

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Kiểm thử Web UI Tự động: Selenium & POM
Buổi 8 / 9 — Môn: Kiểm thử Phần mềm

Nội dung buổi học

- 1 Vị trí của E2E Test trên Testing Pyramid & So sánh công cụ
- 2 Kiến trúc **Selenium WebDriver 4** & Cài đặt môi trường
- 3 Chiến lược định vị phần tử DOM: Locators (id, CSS, XPath)
- 4 Thao tác tương tác Web UI: Form, Dropdown, Checkbox, Alerts
- 5 Cơ chế đồng bộ & Chờ thông minh: Implicit vs **Explicit Wait**
- 6 Xử lý lỗi thường gặp: StaleElementReferenceException
- 7 Mô hình thiết kế **Page Object Model (POM)** chuẩn mực
- 8 Kịch bản E2E hoàn chỉnh trên DigiShield Frontend & Test phân quyền
- 9 Vận hành nâng cao: Chế độ Headless, Screenshot on Failure & CI

Vị trí trong đề cương môn học (Khung 9 buổi)

Buổi Chuyên đề

1-2	C1-C2: Tổng quan, quy trình (STLC)
3	C3: Kiểm thử hộp trắng & JaCoCo
4	C4: Unit Test với JUnit 5 & Mockito
5	C4: Kiểm thử tích hợp (Testcontainers)
6	C5: Kiểm thử hộp đen (EP, BVA, Decision Table)
7	C5: Kiểm thử API (Postman / Newman)
8	C6: Web UI (Selenium & POM) (hôm nay)
9	C7: CI/CD Pipeline & Báo cáo BTL

Chương 6 — Web UI Testing

- 6.1. Selenium 4, Locators (CSS/XPath)
- 6.2. Explicit Waits & chống Flaky Test
- 6.3. Page Object Model (POM) & E2E

Mục tiêu hôm nay

Tự động hóa Chrome click, nhập form, kiểm thử luồng E2E như người dùng thật.

Vị trí của E2E Test trên Testing Pyramid

Đặc điểm tầng đỉnh (E2E Test)

- **Mô phỏng người dùng thật:** Trình duyệt tải HTML, thực thi JS (React), gọi API backend, render CSS.
- **Độ tin cậy nghiệp vụ cao:** Xác nhận toàn bộ hệ thống hoạt động ăn khớp từ frontend tới database.
- **Chi phí cao:** Chạy chậm (gấp 10–100 lần API test), dễ bị ảnh hưởng bởi mạng và thời gian render.

Nguyên tắc thực tế

- **Viết ít nhưng trúng đích:** Không test mọi biến thể dữ liệu biên trên UI (đó là việc của Unit/API test).
- Chỉ dùng E2E cho các **luồng nghiệp vụ sống còn (Happy Paths & Core Flows)**: Đăng nhập, Tạo đơn hàng, Can thiệp giao dịch lừa đảo.

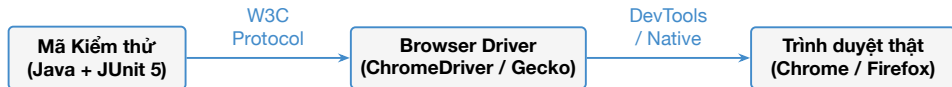
So sánh bộ ba: Selenium vs Cypress vs Playwright

Tiêu chí	Selenium WebDriver	Cypress	Playwright
Ngôn ngữ	Java, Python, C#, JS	Chỉ JavaScript/TS	JS/TS, Python, Java, C#
Kiến trúc	Chuẩn W3C ngoài trình duyệt	Chạy trực tiếp trong DOM	Qua Chrome DevTools Protocol
Hỗ trợ Tab/IFrame	Cực mạnh (đa tab, đa window)	Hạn chế tab/window	Rất tốt
Hệ sinh thái	Lâu đời, tiêu chuẩn doanh nghiệp	Thân thiện Frontend dev	Hiện đại, tốc độ cao
Phù hợp môn học	Hoàn hảo: Đồng bộ Java/JUnit 5 với backend DigiShield	Frontend JS/TS thuần	Dự án đa ngôn ngữ mới

Kiến trúc Selenium WebDriver 4 (Chuẩn W3C)

Cơ chế hoạt động trực tiếp qua W3C WebDriver Standard

Không còn giao thức trung gian JSON Wire Protocol cũ; mã test giao tiếp trực tiếp với Driver theo chuẩn W3C của các trình duyệt hiện đại.



Điểm sáng Selenium 4: Tích hợp sẵn Selenium Manager

Không cần tải thủ công file `chromedriver.exe` hay thiết lập `System.setProperty(...)`. Thư viện tự động phát hiện phiên bản Chrome trên máy và tải driver tương thích hoàn toàn tự động!

Hiểu nhanh kiến trúc (1/2): Ví dụ nhà hàng

Selenium giống một nhà hàng

- **Mã kiểm thử** = Khách gọi món (“bấm nút Đăng nhập”).
- **Browser Driver** = Phục vụ: nhận order, báo lại cho bếp bằng ngôn ngữ bếp hiểu.
- **Trình duyệt** = Bếp: thực sự nấu (click, gõ chữ, tải trang).
- **Chuẩn W3C** = Mẫu phiếu order chung, nên đổi bếp Chrome sang Firefox mà khách không phải học lại.

Hiểu nhanh kiến trúc (2/2): Một lệnh chạy ra sao?

- 1 Java gửi một **HTTP request** (JSON) tới ChromeDriver, ví dụ POST /session/{id}/element.
- 2 ChromeDriver dịch thành lệnh điều khiển Chrome thật.
- 3 Chrome tìm phần tử trong DOM rồi click.
- 4 Kết quả (thành công hoặc lỗi) trả ngược về Java dưới dạng JSON.

Vì vậy `driver.quit()` rất quan trọng

Nếu quên, tiến trình ChromeDriver và Chrome vẫn chạy ngầm, máy chậm dần.

Cài đặt thư viện Selenium (Gradle)

Khai báo trong build.gradle

Chỉ cần 3 dòng: Selenium, JUnit 5 để viết test, AssertJ để viết câu kiểm tra dễ đọc.

```
dependencies {  
    testImplementation 'org.seleniumhq.selenium:selenium-java:4.25.0'  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.2'  
    testImplementation 'org.assertj:assertj-core:3.26.0'  
}
```

Thư viện selenium-java kéo theo toàn bộ driver và công cụ điều khiển trình duyệt.

Vòng đời Test với JUnit 5: Mở và đóng trình duyệt

```
class BaseWebTest {
    protected WebDriver driver;
    @BeforeEach
    void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }
    @AfterEach
    void tearDown() {
        if (driver != null) {
            driver.quit(); // QUAN TRỌNG
        }
    }
}
```

- @BeforeEach: chạy **trước mỗi** test, mở một trình duyệt mới sạch sẽ.
- @AfterEach: chạy **sau mỗi** test, kể cả khi test FAIL.
- driver.quit() đóng trình duyệt và tắt tiến trình ChromeDriver. Quên dòng này là máy đầy Chrome chạy ngầm.

Nhập môn: Một trang web được tạo nên từ gì?

Trang web = các “khối” HTML lồng nhau, mỗi khối gọi là một **phần tử** (element)

Selenium không “nhìn” hình ảnh như con người. Nó đọc cấu trúc HTML, nên ta cần biết cách đọc một thẻ.

`<input type="text" name="username" placeholder="Ten dang nhap">`

Tên thẻ
(loại phần tử)

Thuộc tính
(nhãn mô tả)

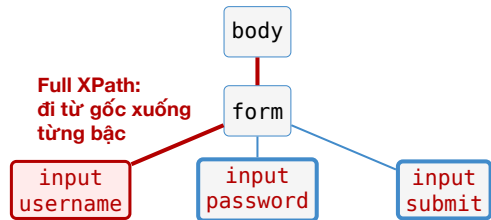
Giá trị
của thuộc tính

Locator là gì? Hãy nghĩ đến địa chỉ nhà

Shipper và Selenium cùng cần “địa chỉ”

- Muốn shipper tìm đúng nhà, ta đưa **thông tin nhận dạng**: số nhà, tên người nhận.
- Muốn Selenium tìm đúng phần tử, ta đưa **locator**: “tìm thẻ có name="username"”.
- Địa chỉ càng đặc trưng thì càng ít giao nhầm nhà.

DOM là gì? Cây gia phả của trang web



Đường đỏ = đường đi của full XPath. Ô đỏ = phần tử cần tìm.

Đọc cây như gia phả

- form là **cha**, ba thẻ input là **con**.
- Ba input là **anh em** ngang hàng.
- **DOM** = “gia phả” mà trình duyệt dựng ra từ HTML. Selenium tìm phần tử trên cây này.

Vì sao điều này quan trọng?

Full XPath = “đi từ ông tổ xuống đúng người”, đổi một đời là lạc. **Locator theo thuộc tính** = “người tên là X”, không phụ thuộc đứng ở đâu.

Họ bộ định vị (Locators) trong Selenium

Nguyên tắc: `driver.findElement(By.method("..."))`

Mọi thao tác tự động hóa bắt đầu bằng việc xác định tọa độ phần tử trên cây DOM.

Loại Locator	Cú pháp Java	Mức ưu tiên
ID	<code>By.id("login-email")</code>	1. Ưu tiên cao nhất
Name	<code>By.name("username")</code>	2. Tốt cho form
CSS Selector	<code>By.cssSelector("#login-btn")</code>	3. Chuẩn mực web
Link Text	<code>By.linkText("Quên mật khẩu?")</code>	4. Tiện lợi cho thẻ <code><a></code>
Class Name	<code>By.className("btn-primary")</code>	5. Dễ trùng lặp
XPath	<code>By.xpath("//a[text()='Xoa']")</code>	6. Phức tạp/Đặc biệt

Làm chủ CSS Selectors trong Automation Test

Tại sao CSS Selector được ưa chuộng?

Cú pháp ngắn gọn, tốc độ tìm kiếm nhanh hơn XPath trên hầu hết trình duyệt, tương thích hoàn toàn với kiến trúc frontend hiện đại.

Mẫu lựa chọn	Ý nghĩa	Ví dụ
#id	Phần tử có id chỉ định	#login-btn
.class	Phần tử có class tương ứng	.btn-danger
[attr='val']	Thuộc tính bằng chính xác	[type='password']
[attr^='val']	Bắt đầu bằng chuỗi con	[id^='input-']
[attr*='val']	Chứa chuỗi con bất kỳ	[class*='active']
parent > child	Con trực tiếp cấp 1	table.soc-grid > tbody
ancestor desc	Con cháu mọi cấp độ	form#login-form button

Làm chủ XPath: Khi nào bắt buộc phải dùng XPath?

Thế mạnh độc quyền của XPath mà CSS Selector không làm được

1. Tìm phần tử theo nội dung văn bản (`text()`).
2. Đi ngược cây DOM từ con lên cha hoặc sang phần tử ngang hàng (Axes).

Tìm theo nội dung chữ:

<code>//button[text()='Dang nhap']</code>	<code><-</code> nút có chứa chữ "Dang nhap"
<code>//a[contains(text(), 'Huong dan')]</code>	<code><-</code> link có chứa "Huong dan"

Đi ngược DOM & trục Axes:

<code>//input[@id='email']/ancestor::form</code>	<code><-</code> từ ô nhập tìm lên the form cha
<code>//td[text()='0123456789']/following-sibling::td/button</code>	<code><-</code> nút nằm ở cột bên cạnh số TK

Cách lấy CSS Selector & XPath bằng Chrome DevTools

4 bước lấy locator của một phần tử

- 1 Mở trang, nhấn **F12** (hoặc chuột phải → **Inspect**).
- 2 Bấm biểu tượng mũi tên (**Ctrl+Shift+C**) rồi click vào phần tử cần tìm.
- 3 Tab **Elements** tô sáng thẻ HTML tương ứng. Đọc id, name, data-testid trước tiên.
- 4 Chuột phải vào thẻ → **Copy** → **Copy selector** (CSS) hoặc **Copy XPath**.

Lưu ý

Selector do Chrome copy thường **quá dài và dễ gãy** (kiểu `div:nth-child(3) > input`). Hãy rút gọn thành `[data-testid='...']` hoặc `#id`.

Kiểm tra selector trước khi viết code

Dùng tab Console của DevTools

- CSS: `$$("[data-testid='login-submit']")`
- XPath: `$x("//button[text()='Đăng nhập']")`

Đọc kết quả trả về (mảng phần tử)

- **1 phần tử**: locator tốt, dùng được.
- **0 phần tử**: sai selector (hoặc phần tử nằm trong iframe, chưa render).
- **Nhiều phần tử**: chưa đủ duy nhất, `findElement` sẽ lấy phần tử đầu tiên.

Mẹo: **Ctrl+F** trong tab Elements cũng nhận cả CSS lẫn XPath và hiện “1 of 1”.

Nhìn trang web, chọn locator (1/2)

Tài khoản văn phòng điện tử

Tên đăng nhập

Mật khẩu

☐ Giữ tôi luôn đăng nhập



Đăng nhập bằng e-mail UTC

Đăng nhập

Bạn quên mật khẩu đăng nhập ?

Phần tử → Locator

- 1 Ô tên đăng nhập
By.name ("username")
 - 2 Ô mật khẩu
By.name ("userpwd")
 - 3 Ô tick “Giữ tôi luôn đăng nhập”
label.check (ô thật bị ẩn)
-

Nhìn trang web, chọn locator (2/2)

Tài khoản văn phòng điện tử

Tên đăng nhập

Mật khẩu

☐ Giữ tôi luôn đăng nhập



Đăng nhập bằng e-mail UTC

Đăng nhập

[Bạn quên mật khẩu đăng nhập ?](#)

Phần tử → Locator

- 4 Nút “Đăng nhập bằng e-mail UTC”
`//a[contains(text(),
'e-mail UTC')]`
 - 5 Nút “Đăng nhập”
`input.submit_login`
 - 6 Link “Bạn quên mật khẩu”
`a[href='/Login/GetPass']`
-

Ví dụ thực tế: HTML trang đăng nhập UTC

```
<form action="/Login" method="post">
  <input type="text"
    name="username"
    placeholder="Ten dang nhap"/>
  <input type="password"
    name="userpwd"
    placeholder="Mat khau"/>
  <input type="checkbox"
    id="persistent"
    name="persistent"/>
  <a class="button" href="...">
    Dang nhap bang e-mail UTC</a>
  <input type="submit"
    class="submit_login"
    value="Dang nhap"/>
</form>
```

Đọc nhanh đoạn HTML

- Nguồn: trang Login của Văn phòng điện tử UTC, đã rút gọn.
- Ô nhập và nút **không có** id.
- Ô nhập có name, nút có class riêng.
- Chỉ ô tick “Ghi nhớ” có id, nhưng JavaScript sẽ ẩn nó đi.

Ví dụ thực tế: Locator cho trang đăng nhập UTC

```
// Ten dang nhap & mat khau: khong co id, dung name
By.name("username")
By.name("userpwd")
// Nut dang nhap: class riêng hoac type
By.cssSelector("input.submit_login")
By.cssSelector("form[action='/Login'] [type='submit']")
// O tick "Ghi nho": o that bi an, click vao o hien thi
By.id("persistent") // de doc isSelected()
By.cssSelector("label.check") // de click
// Nut Google: tim theo chu hien thi (XPath)
By.xpath("//a[contains(text(),'e-mail UTC')]")
```

Bài học: khi không có id/data-testid, thử lần lượt **name** → **class riêng** → **thuộc tính khác** → **XPath theo text**. Tránh XPath tuyệt đối.

Thực hành (1/2): Lấy locator ô username trên trang UTC

- 1 Mở trang /Login, nhấn **F12**.
- 2 Nhấn **Ctrl+Shift+C**, click vào ô “Tên đăng nhập”.
- 3 DevTools tô sáng thẻ `<input ... name="username">`.
- 4 **Đọc thuộc tính**: không có id, không có data-testid, nhưng có name.
- 5 Viết `By.name("username")`.
- 6 Kiểm tra trong Console: `$$("[name='username']")` → ra **1 phần tử**.

Thực hành (2/2): Dùng nút “Copy” của Chrome thì ra gì?

Chuột phải vào thẻ → Copy → **Copy full XPath** (dạng như):

```
/html/body/div/div[1]/div[2]/div[1]/form/input[1]
```

Copy → **Copy selector** (CSS):

```
body > div > div:nth-child(1) > div:nth-child(2)  
  > div:nth-child(1) > form > input[type=text]:nth-child(1)
```

Chạy được, nhưng dài và dễ gãy

Nên đọc thuộc tính rồi tự viết lại ngắn gọn: `By.name("username").`

Vì sao không dùng “Copy full XPath”? (1/2)

Full XPath = đường đi từ gốc đến phần tử

```
/html/body/div/div[1]/div[2]/div[1]/form/input[1]
```

Bám vào vị trí: “div thứ 2 trong div thứ 1 ...”.

Chỉ cần dev:

- thêm một thẻ <div> bọc ngoài (làm lại giao diện),
- chèn thêm một banner/thông báo lên trên,
- đổi thứ tự hai ô input,

là đường đi sai, test báo NoSuchElementException dù chức năng vẫn đúng.

Vì sao không dùng “Copy full XPath”? (2/2)

By.name("username"): bám vào ý nghĩa

Nó nói “ô nào có tên là username”, không quan tâm nằm ở đâu. Layout đổi thế nào cũng vẫn tìm được, miễn là backend còn nhận trường username.

- **Dễ đọc**: nhìn là biết đang tìm gì.
- **Ngắn**, ít lỗi gõ.

Quy tắc nhớ

Locator tốt = **mô tả phần tử là gì**, không phải **nó nằm ở đâu**. Nếu bắt buộc dùng XPath, hãy dùng XPath **tương đối**, như `//input[@name='username']`.

Giải thích chi tiết (1a): By.name

`By.name("username")` tìm ô tên đăng nhập như thế nào?

Selenium tìm phần tử có thuộc tính `name="username"`:

```
<input type="text" name="username" ...>
```

- **Vì sao name ổn định?** Khi bấm Đăng nhập, form gửi lên server cặp `username=...`. Backend đọc đúng tên này nên dev rất ít khi đổi.
- **Vì sao không dùng placeholder?** Đó là chữ hiển thị, sửa câu chữ là test gãy.
- **Vì sao không dùng type="text"?** Trang có thể có nhiều ô text.

Giải thích chi tiết (1b): `By.id` & lưu ý

`By.id("persistent")`: ô tick “Giữ tôi luôn đăng nhập”

- Đây là phần tử **duy nhất** trên trang có id. Khi có id, đó là lựa chọn số 1.
- Nhưng JavaScript **ẩn ô này đi**: chỉ dùng để đọc `isSelected()`. Muốn click thì dùng `label.check` (xem slide Checkbox).

Lưu ý về name

name **không bắt buộc duy nhất** (nhóm radio dùng chung name). Luôn kiểm tra `$$("[name='username']")` ra đúng 1 phần tử.

Giải thích chi tiết (2a): Đọc CSS Selector trên trang UTC

Selector	Đọc từng phần
<code>input.submit_login</code>	Thẻ <code><input></code> và có class <code>submit_login</code> . Viết liền nhau = cả hai điều kiện cùng đúng.
<code>form[action='/Login'] [type='submit']</code>	Thẻ form có <code>action="/Login"</code> , rồi dấu cách = “bên trong nó”, tìm phần tử có <code>type="submit"</code> .
<code>a[href='/Login/GetPass']</code>	Thẻ <code><a></code> có href đúng bằng đường dẫn quên mật khẩu.

Giải thích chi tiết (2b): Cẩn thận khi chọn class

Class button không đủ để tìm nút đăng nhập

Nút “Đăng nhập bằng e-mail UTC” cũng có `class="button"`. Dùng `.button` sẽ ra **sai phần tử**.

Nên chọn class mô tả đúng chức năng

`submit_login` chỉ dùng cho nút Đăng nhập, nên `input.submit_login` tìm ra đúng 1 phần tử.

Mẹo: kiểm tra trong Console bằng `$("#input.submit_login")`, phải ra đúng 1 phần tử.

Giải thích chi tiết (3a): Đọc XPath tìm nút Google

```
//a[contains(text(),'e-mail UTC')]
```

//	Tìm ở mọi vị trí trong trang.
a	Chỉ xét thẻ <a> (link).
[...]	Điều kiện lọc cho thẻ a.
text()	Chữ nằm trực tiếp trong thẻ.
contains(x, y)	Đúng nếu x chứa y.

Kiểm tra trong Console: `$x("//a[contains(text(),'e-mail UTC')]")`
phải ra 1 phần tử.

Giải thích chi tiết (3b): contains và điểm yếu

Vì sao dùng contains, không dùng =?

Chữ trong thẻ thường có thêm khoảng trắng, xuống dòng. So sánh bằng cả câu rất dễ trượt; contains chỉ cần khớp một đoạn đặc trưng.

Điểm yếu cần biết

Locator dựa vào chữ sẽ gãy khi đổi câu chữ hoặc ngôn ngữ. Chỉ dùng khi **không có** id, name, data-testid. Chữ dư khoảng trắng thì dùng `normalize-space(.)`.

Từ điển nhanh cho người mới

Thuật ngữ	Hiểu đơn giản
Phần tử (element)	Một thành phần trên trang: ô nhập, nút bấm, link, bảng.
Thẻ (tag) / Thuộc tính	Tên loại phần tử (input, a, button) / nhãn mô tả gắn vào nó (id, name, class).
DOM	Cây cha-con của các phần tử mà trình duyệt dựng ra.
Locator	“Địa chỉ” cho Selenium biết cần tìm phần tử nào.
CSS Selector	Cách viết địa chỉ theo thẻ, class, thuộc tính. Ngắn gọn.
XPath	Cách viết địa chỉ như đường đi trên cây; còn tìm được theo chữ hiển thị.
Driver	Chương trình trung gian giúp Java điều khiển trình duyệt.
Flaky test	Test lúc qua lúc rớt dù code không đổi.

Các chống mẫu (Anti-patterns) khi chọn Locator

Nên tránh tuyệt đối

- **XPath tuyệt đối:**

`/html/body/div[1]/div[2]/div/form/div[3]/input`

→ Layout đổi thêm 1 thẻ `<div>` là gây toàn bộ test!

- **Class tự sinh:** `.css-19zpn8d`,
`.sc-bczRLJ`

→ Hash CSS đổi sau mỗi lần build Webpack/Vite.

- **Index cứng:** `//table/tr[5]/td[2]`
→ Dữ liệu bảng thay đổi thứ tự là test đọc sai dòng.

Best Practice: data-testid

- Quy ước với nhóm Dev gắn thuộc tính riêng cho test:
`data-testid="login-submit"`
- Tách rời giữa **Style hiển thị** và **Hành vi test**.
- Dev thoải mái đổi Tailwind/Bootstrap class mà test suite không hề bị ảnh hưởng!

Thao tác cơ bản (1): Mở trang, nhập liệu, bấm nút

Ví dụ trên trang `vanphongdientu.utc.edu.vn/Login`

```
// 1. Điều hướng tới trang đang nhập
driver.get("https://vanphongdientu.utc.edu.vn/Login");

// 2. Nhập liệu (luôn clear trước khi gõ text mới)
WebElement username = driver.findElement(By.name("username"));
username.clear();
username.sendKeys("sinhvien01");
driver.findElement(By.name("userpwd")).sendKeys("matkhau123");

// 3. Bấm nút "Đăng nhập"
WebElement submitBtn =
    driver.findElement(By.cssSelector("input.submit_login"));
submitBtn.click();
```

Thao tác cơ bản (2): Đọc dữ liệu & kiểm tra trạng thái

```
// 4. Đọc thuộc tính trong HTML
String hint = username.getAttribute("placeholder"); // "Tên đang nhập"
String label = submitBtn.getAttribute("value");      // "Đăng nhập"
String forgot = driver.findElement(
    By.cssSelector("a[href='/Login/GetPass']")).getText();

// 5. Kiểm tra trạng thái phần tử
boolean isVisible = submitBtn.isDisplayed();
boolean isEnabled = submitBtn.isEnabled();
```

- `getText()`: chữ nhìn thấy. `getAttribute()`: giá trị thuộc tính. Nút `<input>` không có text, chữ “Đăng nhập” nằm ở value.

Xử lý Dropdown (<select>)

Dùng lớp hỗ trợ Select của Selenium

```
WebElement dropdownEl = driver.findElement(By.id("risk-level"));
Select riskSelect = new Select(dropdownEl);

// 1. Chọn theo chu hiện thị trên UI
riskSelect.selectByVisibleText("Rui ro cao (HIGH)");

// 2. Chọn theo giá trị thuộc tính value
riskSelect.selectByValue("HIGH_RISK");

// 3. Kiểm tra giá trị đang được chọn
String selectedText = riskSelect.getFirstSelectedOption().getText();
assertThat(selectedText).contains("HIGH");
```

Xử lý Checkbox: Ô “Giữ tôi luôn đăng nhập” (trang UTC)

Bây thực tế: ô checkbox thật bị JavaScript **ẩn đi**

Trang UTC đặt `display:none` cho `<input id="persistent">` và vẽ một ô giả `<label class="check">`. Gọi `click()` lên ô bị ẩn sẽ ném `ElementNotInteractableException`.

```
// 0 that (bi an): chỉ dùng để đọc trạng thái
WebElement checkbox = driver.findElement(By.id("persistent"));
// 0 hiển thị: dùng để click
WebElement fakeBox = driver.findElement(By.cssSelector("label.check"));
if (!checkbox.isSelected()) {
    fakeBox.click();
}
assertThat(checkbox.isSelected()).isTrue();
```

Selenium chỉ click được thứ **người dùng nhìn thấy**. Hỏi `isSelected()` trước để không bỏ tick nhầm.

Xử lý JavaScript Alert / Confirm Dialog

Hộp thoại của trình duyệt không nằm trong DOM

Không dùng `findElement` được, phải “chuyển ngữ cảnh” sang Alert.

```
// Chuyển ngữ cảnh sang Alert  
Alert alert = driver.switchTo().alert();  
  
// Đọc thông báo  
System.out.println("Nội dung alert: " + alert.getText());  
  
// Bấm nút OK (Accept)  
alert.accept();  
  
// Hoặc bấm nút Cancel (Dismiss)  
// alert.dismiss();
```

Chuyển Tab / Cửa sổ mới (trang UTC)

Link “Trung tâm trợ giúp” có target="_blank" nên mở tab mới, nhưng Selenium vẫn đứng ở tab cũ. Phải tự chuyển theo mã tab (window handle).

```
String mainTab = driver.getWindowHandle();
driver.findElement(
    By.cssSelector("a[href='http://hotrokythuat.utc.edu.vn']")).click();

for (String tab : driver.getWindowHandles()) {
    if (!tab.equals(mainTab)) {
        driver.switchTo().window(tab);
        break;
    }
}
assertThat(driver.getCurrentUrl()).contains("hotrokythuat.utc.edu.vn");
driver.close(); // Đóng tab trợ giúp
driver.switchTo().window(mainTab); // Về trang đang nhập
```

Căn bệnh Flaky Test trong kiểm thử Web UI

Vì sao test Web UI hay chậm chạp?

- Web hiện đại (React/Next.js) là Single Page Application (**SPA**).
- Sau khi click button, trang **không reload** mà gọi AJAX/fetch ngầm; DOM cập nhật bất đồng bộ.
- Mã test chạy quá nhanh: Kiểm tra phần tử ngay khi nó chưa kịp render xong → ném lỗi `NoSuchElementException`!

Chống mẫu: `Thread.sleep()`

- Dev chèn bừa `Thread.sleep(5000)` (chờ 5 giây).
- **Hậu quả:** Nếu mạng nhanh vẫn phí mất 5 giây; nếu mạng lag mất 5.1 giây thì test vẫn FAIL!
- $100 \text{ test case} \times 5\text{s} = 8 \text{ phút chờ vô ích!}$

Giải pháp chuẩn mực: Sử dụng cơ chế Chờ đợi tường minh (Explicit Wait)!

Implicit Wait (Chờ ngầm định)

Cài đặt 1 lần cho toàn bộ driver, áp dụng cho mọi lệnh `findElement`

```
// Neu chua thay, driver thu lai lien tuc toi da 10 giay  
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Nhược điểm

Chỉ kiểm tra phần tử **có mặt trong DOM**, không biết nó đã **bấm được** hay đang bị ẩn. Ví dụ ô `#persistent` trên trang UTC có trong DOM nhưng bị ẩn, implicit wait vẫn coi là “tìm thấy”.

Explicit Wait (WebDriverWait): Cách chuẩn mực

Chờ đúng điều kiện cần, xong là chạy tiếp ngay

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

// Cho tôi khi nút "Đăng nhập" hiện ra và bấm được
WebElement btn = wait.until(ExpectedConditions
    .elementToBeClickable(By.cssSelector("input.submit_login")));
btn.click();

// Cho tôi khi trình duyệt rời khỏi trang /Login
wait.until(ExpectedConditions.not(
    ExpectedConditions.urlContains("/Login")));
```

Các điều kiện chờ phổ biến trong ExpectedConditions

Phương thức	Ý nghĩa áp dụng
<code>presenceOfElementLocated(locator)</code>	Element đã có trong DOM (chưa chắc đã hiển thị).
<code>visibilityOfElementLocated(locator)</code>	Element có trong DOM và kích thước > 0 , hiển thị trên màn hình .
<code>elementToBeClickable(locator)</code>	Element hiển thị và ở trạng thái enable (sẵn sàng nhận click).
<code>textToBePresentInElement(el, text)</code>	Chờ text bên trong element cập nhật đúng giá trị mong muốn.
<code>invisibilityOf(element)</code>	Chờ thanh loading hoặc popup thông báo đóng lại hoàn toàn.
<code>urlContains(subString)</code>	Chờ trình duyệt hoàn tất chuyển trang (URL thay đổi sau login).

Xử lý ngoại lệ StaleElementReferenceException

Bản chất lỗi Stale Element trong ứng dụng React / Vue

React **re-render** $\Rightarrow$ phần tử cũ bị xóa, thay bằng phần tử mới $\Rightarrow$ biến `WebElement` giữ từ trước trở vào phần tử “đã chết” (stale).

Đoạn mã gây lỗi:

```
WebElement btn = driver.findElement(By.id("refresh-btn"));  
// ... React re-render DOM ở giữa ...  
btn.click(); // NEM: StaleElementReferenceException
```

Cách xử lý: tìm lại phần tử ngay lúc cần dùng

```
wait.until(ExpectedConditions.refreshed(  
    ExpectedConditions.elementToBeClickable(By.id("refresh-btn"))  
)).click();
```

Vì sao cần Page Object Model (POM)?

Khi KHÔNG dùng POM

- `Selector By.id("txtEmail")` bị copy-paste rải rác ở 20 file test khác nhau.
- Khi dev frontend đổi `id="txtEmail"` thành `id="username"` $\Rightarrow$ Tester phải đi **sửa thủ công cả 20 file test!**
- Code test trộn lẫn giữa logic tương tác DOM (`sendKeys`, `click`) và logic kiểm tra nghiệp vụ (`assertThat`).

Giải pháp POM (Martin Fowler)

- Mỗi trang Web / Component được mô hình hóa thành một **Java Class** riêng biệt.
- Class này đóng gói toàn bộ:
 - ➊ **Locators:** Địa chỉ các phần tử DOM trên trang.
 - ➋ **Actions:** Các hành vi người dùng thực hiện (nhập, click).
- File test chỉ việc gọi các method nghiệp vụ trong sáng.

3 Nguyên tắc vàng khi thiết kế Page Object

Nguyên tắc 1: Page Object KHÔNG chứa Assertions

Nhiệm vụ của Page Object là cung cấp dịch vụ giao diện (nhập, bấm, lấy text). Việc phán xét đúng/sai thuộc về Test Class (ngoại trừ assert trạng thái trang đã sẵn sàng).

Nguyên tắc 2: Mọi Locators phải được đóng gói private

Không bao giờ để file test gọi `loginPage.usernameField.click()`. Tất cả selector phải được giấu kín sau các method nghiệp vụ như `loginPage.loginAs("...", "...")`.

Nguyên tắc 3: Fluent Navigation (Trả về Page kế tiếp)

Hành động chuyển trang nên trả về instance của trang đích:

```
HomePage home = loginPage.loginAs("sinhvien01", "matkhau123");
```

Cấu trúc thư mục chuẩn cho dự án E2E Test

Tách riêng Page Objects và Test Scripts

```
digishield-e2e/src/test/java/.../e2e/
|-- base/
|   |-- BaseTest.java          // Khởi tạo WebDriver, timeout
|-- pages/                     // CAC PAGE OBJECTS
|   |-- BasePage.java          // Thao tác chung: wait, click, type
|   |-- LoginPage.java         // Form đăng nhập
|   |-- WatchlistPage.java     // Quản lý tài khoản nghỉ ngơi
|   |-- DashboardPage.java     // Menu, header, thông tin user
|-- tests/                     // CAC TEST SCRIPTS (JUNIT 5)
    |-- LoginE2ETest.java
    |-- WatchlistE2ETest.java
```

Tầng cơ sở: BasePage.java (1/2)

Lớp cha cho mọi Page Object: giữ driver và bộ chờ

```
public abstract class BasePage {  
    protected WebDriver driver;  
    protected WebDriverWait wait;  
  
    public BasePage(WebDriver driver) {  
        this.driver = driver;  
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
    }  
}
```

Tầng cơ sở: BasePage.java (2/2)

Các hàm tiện ích luôn chờ trước khi thao tác

```
protected void click(By locator) {  
    wait.until(ExpectedConditions.elementToBeClickable(locator))  
        .click();  
}  
  
protected void type(By locator, String text) {  
    WebElement el = wait.until(ExpectedConditions  
        .visibilityOfElementLocated(locator));  
    el.clear();  
    el.sendKeys(text);  
}  
  
protected String getText(By locator) {  
    return wait.until(ExpectedConditions  
        .visibilityOfElementLocated(locator)).getText();  
}  
}
```

Cài đặt Page Object: LoginPage.java (1/2)

Bước 1: Khai báo locator của trang UTC, để private

```
public class LoginPage extends BasePage {  
    public static final String URL =  
        "https://vanphongdientu.utc.edu.vn/Login";  
  
    private final By usernameField = By.name("username");  
    private final By passwordField = By.name("userpwd");  
    private final By loginButton    = By.cssSelector("input.submit_login");  
  
    public LoginPage(WebDriver driver) {  
        super(driver);  
    }  
}
```

Cài đặt Page Object: LoginPage.java (2/2)

Bước 2: Method nghiệp vụ, trả về trang kế tiếp

```
public LoginPage open() { driver.get(URL); return this; }

public HomePage loginAs(String username, String password) {
    type(usernameField, username);
    type(passwordField, password);
    click(loginButton);
    return new HomePage(driver);    // Trang sau khi đang nhập
}

public boolean isOnLoginPage() {
    return driver.getCurrentUrl().contains("/Login");
}
}
```

DigiShield: DashboardPage.java

```
public class DashboardPage extends BasePage {
    private final By userEmailLabel = By.id("user-email");
    private final By watchlistMenu = By.cssSelector("a[href='/watchlist']");
    public DashboardPage(WebDriver driver) { super(driver); }
    public String getLoggedInUserEmail() { return getText(userEmailLabel); }
    public boolean isWatchlistMenuVisible() {
        return wait.until(ExpectedConditions
            .visibilityOfElementLocated(watchlistMenu)).isDisplayed();
    }
    public WatchlistPage goToWatchlistPage() {
        click(watchlistMenu);
        return new WatchlistPage(driver);
    }
}
```

Mã kiểm thử tinh gọn khi áp dụng POM (1/2)

File test chỉ chứa nghiệp vụ và Assertions, không có selector nào

Tài khoản thật lấy từ biến môi trường, **không bao giờ** viết mật khẩu vào code.

```
@Test
@DisplayName("Đăng nhập thành công -> rời khỏi trang Login")
void login_whenValidCredentials_leavesLoginPage() {
    LoginPage loginPage = new LoginPage(driver).open();
    loginPage.loginAs(System.getenv("UTC_USER"),
                      System.getenv("UTC_PASS"));
    new WebDriverWait(driver, Duration.ofSeconds(10))
        .until(ExpectedConditions.not(
            ExpectedConditions.urlContains("/Login")));
    assertThat(loginPage.isOnLoginPage()).isFalse();
}
```

Mã kiểm thử tinh gọn khi áp dụng POM (2/2)

Test trường hợp sai mật khẩu

```
@Test
@DisplayName("Sai mat khau -> van o lai trang Login")
void login_whenWrongPassword_staysOnLoginPage() {
    LoginPage loginPage = new LoginPage(driver).open();

    loginPage.loginAs("sinhvien01", "SaiMatKhau");

    assertThat(loginPage.isOnLoginPage()).isTrue();
}
```

Đọc test như đọc kịch bản: mở trang → đăng nhập → kiểm tra kết quả.

Khái niệm PageFactory & @FindBy

```
public class LoginPageFactory {  
    @FindBy(name = "username")  
    private WebElement usernameInput;  
  
    @FindBy(name = "userpwd")  
    private WebElement passwordInput;  
  
    @FindBy(css = "input.submit_login")  
    private WebElement submitBtn;  
  
    public LoginPageFactory(WebDriver d) {  
        PageFactory.initElements(d, this);  
    }  
}
```

Nhận định thực tế

- **Ưu điểm:** code ngắn, nhìn giống Spring DI.
- **Nhược điểm:** dễ gặp StaleElementReferenceException khi trang re-render.
- **Khuyến nghị:** đa số dự án hiện nay dùng **By locator + Explicit Wait** để kiểm soát thời gian chờ.

Kịch bản E2E hoàn chỉnh: Thêm tài khoản lừa đảo (1/2)

Org Admin đăng nhập → vào Watchlist → thêm tài khoản → xác nhận trong bảng

```
@Test
@DisplayName("E2E: Thêm tài khoản nghi vấn vào danh sách đen")
void watchlist_whenAddScamAccount_displaysInTable() {
    // Du lieu test duy nhat moi lan chay
    String scamAccount = "9999" + System.currentTimeMillis();

    // Helper trong BaseTest: dang nhap DigiShield voi vai tro Org Admin
    DashboardPage dashboard = loginAsOrgAdmin();
    WatchlistPage watchlistPage = dashboard.goToWatchlistPage();
```

Kịch bản E2E hoàn chỉnh: Thêm tài khoản lừa đảo (2/2)

```
// Thực hiện thêm mới
watchlistPage.clickAddNew();
watchlistPage.fillAccountDetails(scamAccount,
    "Tai khoan lua dao mao danh Viettel");
watchlistPage.submitForm();

// Kiểm tra kết quả trên bảng
assertThat(watchlistPage.isAccountPresentInTable(scamAccount))
    .isTrue();
assertThat(watchlistPage.getRiskBadgeFor(scamAccount))
    .isEqualTo("HIGH_RISK");
}
```

Mã test đọc như bản mô tả thao tác của người dùng, nên rất dễ bảo trì.

Kiểm thử Phân quyền Giao diện (RBAC UI Test)

ANALYST chỉ được xem, không được thấy menu “Cấu hình Hệ thống” của ADMIN

```
@Test
@DisplayName("Phan quyen: Analyst không thay menu Cau hinh")
void dashboard_whenLoginAsAnalyst_hidesAdminSettingsMenu() {
    loginAsAnalyst();    // Helper đang nhập DigiShield vai trò Analyst

    // Phan tu KHONG ton tai -> danh sach rong
    boolean hasSettings = driver
        .findElements(By.id("menu-admin-settings")).size() > 0;
    assertThat(hasSettings).isFalse();
}
```

Mẹo: kiểm tra phần tử **không có** trên trang bằng `findElements().size() == 0`.
`findElement()` sẽ ném exception làm test fail.

Vận hành không giao diện: Chế độ Headless

Máy chủ CI (GitHub Actions, Linux) không có màn hình → Chrome thường sẽ crash

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");           // Chạy không hiển thị cửa sổ
options.addArguments("--window-size=1920,1080");  // Có định kích thước
options.addArguments("--no-sandbox");              // Bắt buộc trong container
options.addArguments("--disable-dev-shm-usage");

WebDriver driver = new ChromeDriver(options);
```

Ưu điểm

Chạy được trên máy chủ không màn hình, thường nhanh hơn, và không chiếm màn hình máy cá nhân khi bộ test đang chạy.

Tự động chụp ảnh màn hình khi test FAIL

JUnit 5 Extension: test vừa gãy là lưu ảnh ngay khoảnh khắc đó

```
public class ScreenshotWatcher implements AfterTestExecutionCallback {
    @Override
    public void afterTestExecution(ExtensionContext ctx) throws Exception {
        if (ctx.getExecutionException().isPresent()) { // Test FAIL
            File src = ((TakesScreenshot) BaseTest.getDriver())
                .getScreenshotAs(OutputType.FILE);
            Path dest = Path.of("build/screenshots/"
                + ctx.getDisplayName() + ".png");
            Files.copy(src.toPath(), dest,
                StandardCopyOption.REPLACE_EXISTING);
        }
    }
}
```

Đăng ký trên test class bằng `@ExtendWith(ScreenshotWatcher.class)`.

Quản lý Flaky Test & Cô lập Dữ liệu

Nguyên nhân Flaky hàng đầu

- **Dữ liệu rác (Data Collision):** Test 1 tạo user test@utc.edu.vn, Test 2 chạy lại ném lỗi “Tài khoản đã tồn tại”.
- **Thứ tự thực thi:** Test B ngầm phụ thuộc vào việc Test A phải chạy trước.
- **Hiệu ứng Animation:** React fade-in mất 300ms, click trúng khi element chưa ổn định vị trí.

Giải pháp cô lập dữ liệu

- **Dynamic Test Data:** Luôn dùng timestamp hoặc UUID cho dữ liệu tạo mới:

```
String acc = "ACC" +  
    UUID.randomUUID();
```
- **API Data Seeding / Cleanup:** Trước khi test UI, gọi trực tiếp API backend để xóa/tạo sạch dữ liệu cho test đó.
- **Độc lập tuyệt đối:** Mỗi test tự chuẩn bị dữ liệu cho chính mình.

Bài tập nộp sau buổi 8

Nội dung bài tập thực hành (LMS)

- Cài đặt hoàn chỉnh LoginPage cho DigiShield (hoặc web đề tài tự do) bằng Page Object Model.
- Viết 2 kịch bản: 1 ca đăng nhập thành công và 1 ca kiểm tra lỗi validation khi bỏ trống mật khẩu.
- Áp dụng Explicit Wait với ExpectedConditions, tuyệt đối không dùng `Thread.sleep()`.
- Cấu hình chạy thử nghiệm ở chế độ `--headless` trên môi trường dòng lệnh.

Quy cách nộp

Nộp mã nguồn test lên hệ thống LMS trước buổi học tiếp theo.

Chuẩn bị Buổi 9: CI/CD & Bảo vệ Bài tập lớn

Nội dung Buổi 9 (Buổi cuối cùng trong chương trình!)

- Tự động hóa toàn diện: Xây dựng đường ống **CI/CD Pipeline với GitHub Actions**.
- Tích hợp tự động chạy Unit Test → JaCoCo Coverage Check → Newman API Test → Selenium Headless Test trên mỗi lệnh `git push`.
- Tổng kết toàn bộ quy trình kiểm thử chất lượng phần mềm (STLC).
- **Báo cáo và bảo vệ Bài tập lớn BTL DigiShield** của các nhóm!

Yêu cầu hoàn thiện gấp

- Hoàn thiện toàn bộ code kiểm thử (Unit, Integration, API, Web UI) trên repository nhóm.
- Tổng hợp báo cáo BTL dạng PDF và slide thuyết trình nhóm (thời lượng 10 phút/nhóm).

Tóm tắt buổi 8 (1/2): Chọn Locator

Thứ tự ưu tiên khi chọn locator

id / data-testid → name → class riêng / thuộc tính khác (CSS) → XPath theo text

- Locator tốt **mô tả phần tử là gì**, không phải nó nằm ở đâu. **Không dùng full XPath.**
- Lấy locator bằng **DevTools** (F12, Ctrl+Shift+C), đọc thuộc tính rồi tự viết lại ngắn gọn.
- Luôn kiểm tra trong Console: `$$("css")` hoặc `$x("xpath")` phải ra **đúng 1 phần tử**.
- Selenium chỉ click được thứ **người dùng nhìn thấy** (bấy checkbox bị ẩn trên trang UTC).

Tóm tắt buổi 8 (2/2): WebDriver & POM

Selenium WebDriver Core:

- Chuẩn giao tiếp trực tiếp W3C hiện đại.
- **Explicit Wait** giải quyết triệt để vấn đề bất đồng bộ trong các ứng dụng React/SPA.
- Chế độ **Headless** giúp test chạy được trên máy chủ CI không có màn hình.

Page Object Model (POM):

- Tách rời giao diện UI và kịch bản kiểm thử.
- Frontend đổi giao diện chỉ cần sửa Page Object, không phải sửa từng file test.
- Code test sáng rõ, mang tính diễn đạt nghiệp vụ cao.
- Bổ sung **TakesScreenshot** giúp định vị lỗi tức thì.

Tài liệu tham khảo

- [1] Selenium WebDriver Documentation, <https://www.selenium.dev/documentation/webdriver/>
- [2] Martin Fowler. *Page Object Feature*, <https://martinfowler.com/bliki/PageObject.html>
- [3] W3C WebDriver Specification, <https://www.w3.org/TR/webdriver2/>
- [4] Glenford J. Myers. *The Art of Software Testing*, 2nd ed., John Wiley & Sons, 2004.
- [5] DigiShield Frontend & E2E Testing Suite: frontend/, e2e/.